

Java Library Management System

Project Report

Group 5

Student Name	USP Number
Theo Urbano Gaudencio de Sene	12558717

[https://github.com/Theo-U-
usp/Trabalho_ProgramacaoOrientadaObjetos](https://github.com/Theo-U-usp/Trabalho_ProgramacaoOrientadaObjetos)

1. Requirements

The assignment required a desktop library management system with book management, patron management, loan management, reports, graphical user interface, data persistence, user authentication and exception handling. The current implementation addresses these requirements using Java, Swing, file I/O and JUnit.

Requirement	Implementation	Status
Book Management	Admin can add, edit, delete and search books. Each book includes title, author, ISBN, genre, year and available copies.	Implemented
Patron Management	Admin can add, edit, delete and search patrons. Each patron includes name, contact, ID, current loan state, total fine and borrowing history.	Implemented
Loan Management	Admin and Librarian can check out and check in books. Due date is automatically set to 14 days after checkout.	Implemented
Overdue Fines	LoanManagement calculates overdue days. Fine is calculated as overdue days multiplied by R\$1.20.	Implemented
Reports	The system lists current loans, overdue loans with patron contact details, and patron borrowing history.	Implemented

GUI Design	Swing GUI with LoginScreen, AdminScreen, LibrarianScreen and tabbed views for Books, Patrons and Loans.	Implemented
Search Bars and Tables	Books and patrons can be searched through radio-button search options and JTable result views.	Implemented
Data Persistence	Books, patrons and active loans are saved to and loaded from library.dat using Java serialization.	Implemented
Consistency	The GUI prevents deleting books or patrons that are referenced by active loans.	Implemented
Authentication	Login screen with hardcoded credentials and role selection for Admin and Librarian.	Implemented
Exception Handling	Invalid numeric input, invalid email, invalid ISBN/ID and file errors are handled with user-facing dialogs.	Implemented

Additional Implementation Requirements

The implementation added the following constraints: ISBN and patron ID are stored as String values but must contain only numeric characters; the application saves data automatically after changes; both user roles use the same library.dat file; and the system includes a logout option from both screens.

2. Project Description

The project follows a simple object-oriented structure. Entity classes store data, management classes contain business rules, screen classes handle user interaction, and LibraryData acts as a persistence container. The application starts in LibrarySystem, opens LoginScreen, and then opens either AdminScreen or LibrarianScreen depending on the selected role.

System Architecture

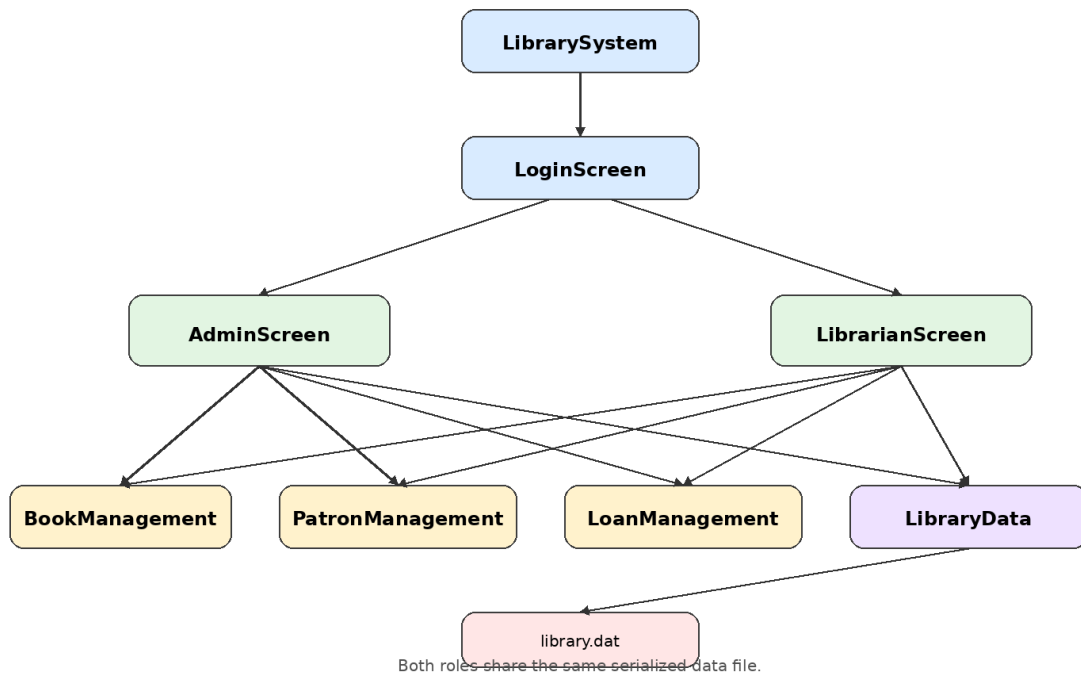


Figure 1. High-level architecture of the application.

Class Relationship Diagram

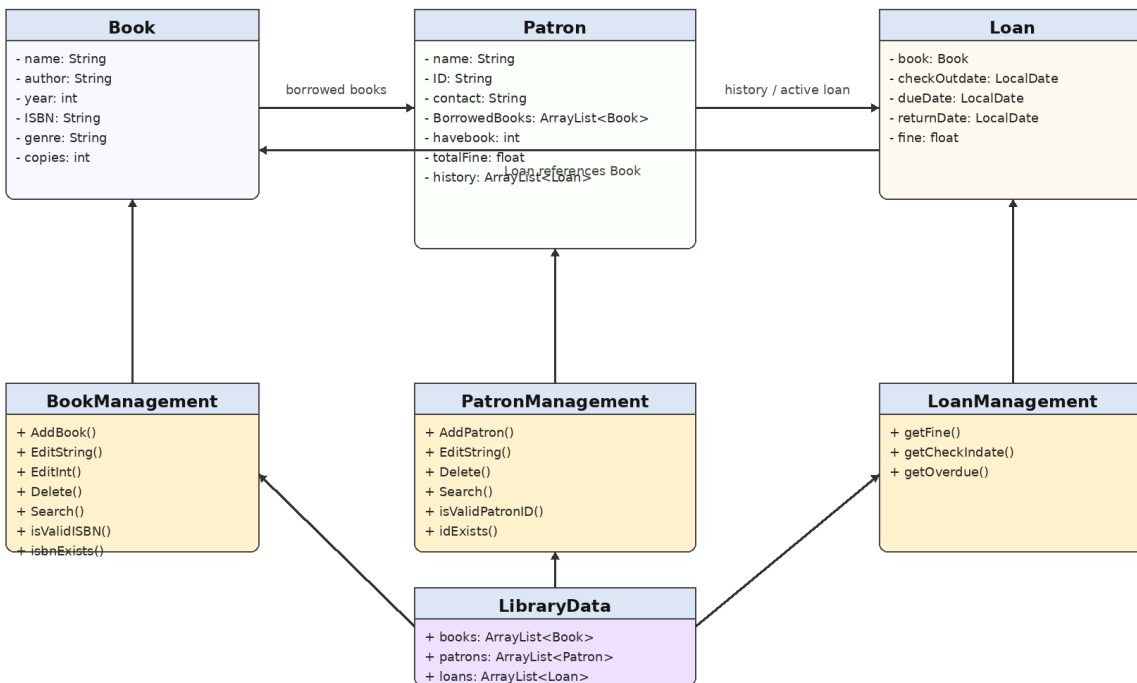


Figure 2. Main classes and relationships.

Role Separation

The Administrator role has full access to books, patrons and loans. The Librarian role is designed for daily circulation tasks and can search records, view history and manage checkouts/checkins, but cannot add, edit or delete books or patrons.

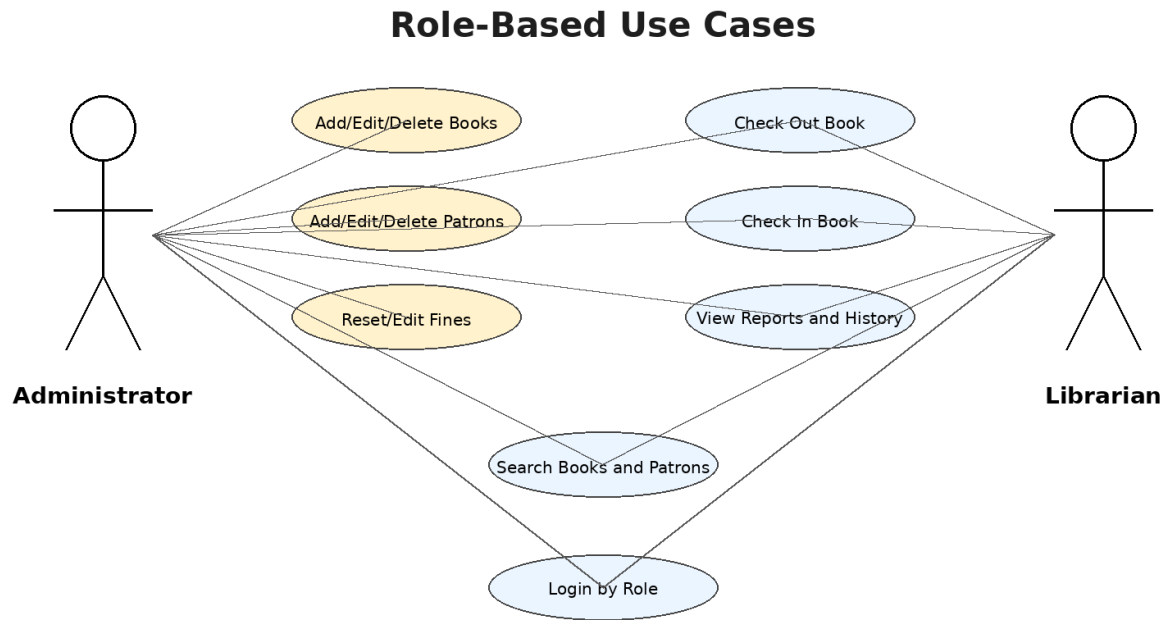


Figure 3. Use cases available to each role.

Loan Workflow

The checkout process verifies that the patron does not already have a book and that the selected book has available copies. The system then creates a Loan object, sets the due date to fourteen days later, updates the patron and book states, and saves the data. During check-in, the system confirms the borrowed book, calculates overdue days and fine, updates the return date, restores book availability and saves the data.

Loan Lifecycle

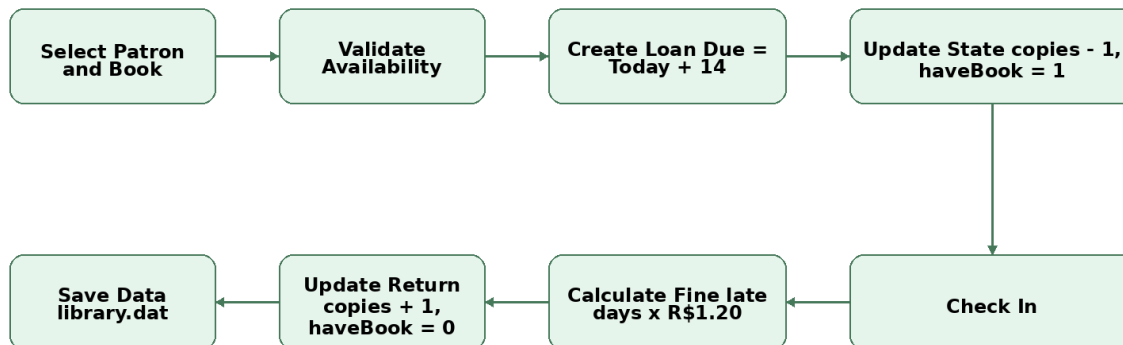


Figure 4. Checkout and check-in lifecycle.

3. Comments About the Code

The code is organized into entity, management, GUI and persistence classes. BookManagement centralizes book operations and ISBN validation. PatronManagement centralizes patron operations and ID validation. This avoids duplicating validation logic inside the GUI. AdminScreen and LibrarianScreen are responsible mainly for user interaction, dialogs, tables and calls to business methods.

Class	Responsibility
LibrarySystem	Starts the application and applies the system look and feel.
LoginScreen	Authenticates users and opens the correct screen.
AdminScreen	Allows full management of books, patrons and loans.
LibrarianScreen	Allows daily loan operations and searches without administrative CRUD actions.
Book	Stores book information.
BookManagement	Handles book operations, searches and ISBN validation.
Patron	Stores patron information, borrowed books and loan history.
PatronManagement	Handles patron operations, searches and ID

	validation.
Loan	Stores loan information and return/fine data.
LoanManagement	Calculates overdue days.
LibraryData	Serializable wrapper used to persist data.
LibraryTests	JUnit test suite.

4. Test Plan

The project uses JUnit 4 for automated tests. The test suite focuses mainly on business logic because GUI dialogs are harder to test automatically. However, simple Swing tests were added to verify that the main screens and important buttons are present.

Test Area	Examples	Expected Result
Book tests	Create, edit, delete, search, ISBN validation, duplicate ISBN	Book state and search results are correct.
Patron tests	Create, edit, delete, search, ID validation, duplicate ID	Patron state and search results are correct.
Loan tests	Create loan, overdue calculation, manual checkout/check-in logic	Due dates, fines and availability updates are correct.
Persistence tests	Create LibraryData with lists	Lists are stored in the data container.
Swing tests	Open Login, Admin and Librarian screens; locate buttons/radio buttons	Screens and expected components exist.

5. Test Results

The implemented tests passed successfully in VS Code using JUnit 4. The screenshot coverage information shows high coverage for data and management classes. GUI classes have lower coverage because many operations depend on JOptionPane dialogs, which are not ideal for automated unit testing.

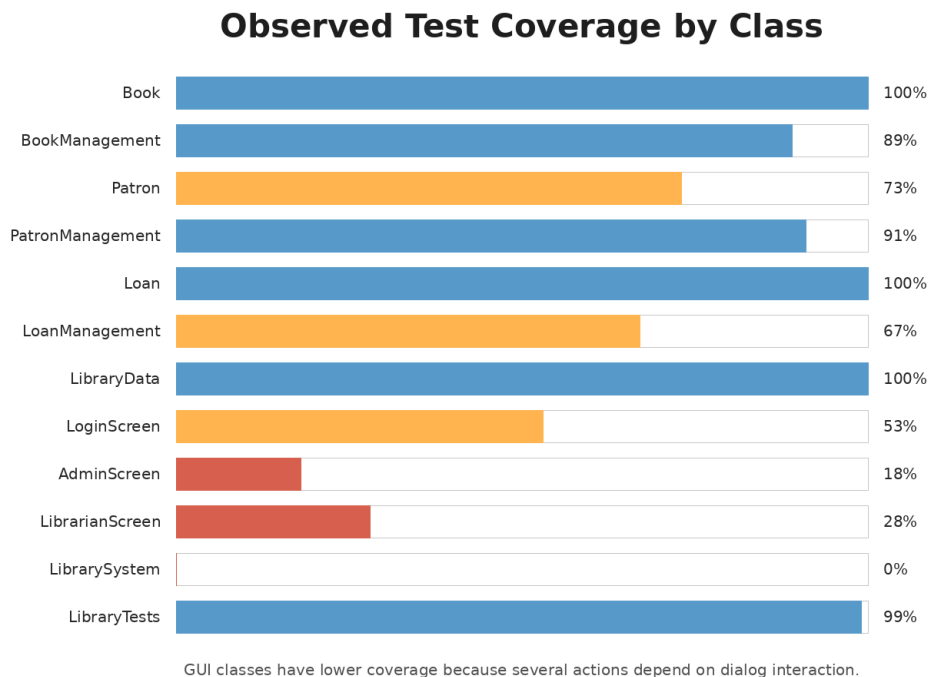


Figure 5. Observed coverage from the current project.

Coverage Interpretation

Book.java, Loan.java and LibraryData.java reached 100% coverage. BookManagement and PatronManagement also reached high coverage because their validation and search methods are directly tested. AdminScreen and LibrarianScreen have lower coverage because complete GUI behavior would require more advanced Swing UI testing tools.

6. Build Procedures

The project is organized as a simple Java project in VS Code. It contains a src folder for Java source files, a lib folder for JUnit dependencies, and a .vscode folder for project settings.

Project structure:

```
PROJETO/
├── .vscode/
│   └── settings.json
├── lib/
│   ├── junit-4.13.2.jar
│   └── hamcrest-core-1.3.jar
└── src/
    ├── AdminScreen.java
    ├── Book.java
    ├── BookManagement.java
    ├── LibrarianScreen.java
    ├── LibraryData.java
    ├── LibrarySystem.java
    └── LibraryTests.java
```

```
| |— Loan.java
| |— LoanManagement.java
| |— LoginScreen.java
| |— Patron.java
| |— PatronManagement.java
|— credential.txt
|— library.dat
```

Steps to build and run:

1. Install Java JDK 17 or newer.
2. Install Visual Studio Code.
3. Install the Extension Pack for Java.
4. Open the project folder in VS Code.
5. Place junit-4.13.2.jar and hamcrest-core-1.3.jar inside the lib folder.
6. Ensure .vscode/settings.json references lib/**/*.jar.
7. Run LibrarySystem.java to start the application.
8. Use the login screen:
 - Admin: username admin, password 1234.
 - Librarian: username librarian, password 5678.
9. Run LibraryTests.java to execute JUnit tests.

7. Problems

The main difficulties were managing Swing dialog behavior, maintaining consistency after changing ISBN and patron ID from int to String, avoiding duplicated validation logic, preventing deleted records from being referenced by active loans, and keeping serialization compatible after class changes. GUI testing also presented limitations because JOptionPane dialogs can block automated tests.

8. Comments

The project implements a functional desktop library management system with authentication, role-based access, persistence, validation, reports and automated tests. Future improvements could include a database instead of serialization, more advanced GUI testing, stronger password storage, and true table sorting using JTable row sorters.